

DirectX 12 엔진 포트폴리오

깃허브 소스 코드 + 엔진에 추가한 기능 설명

<https://github.com/hoya1215/DirectX12-Engine.git>

의도 : C++ 실력과 설계 구조 능력을 키우고 전에 잠시 놓아두었던 DirectX12에 대해서 다시 도전하기 위해 시작했습니다.

목표 : 기본적인 C++ 내용을 실제로 많이 사용하면서 부족한 부분에 대해서 보충을 하고 현대 C++ 내용도 연습을 하면서 더 깊게 언어 능력을 향상 시키고 구조를 설계하는 능력을 키우기

DirectX12 가 어떻게 돌아가는지 이해하고 DirectX11 에서 구현했던 그래픽스 기술 이외에 다른 기술들도 구현하기

얻게 된 결과 : 잘못된 공부 방법으로 DirectX12 를 했었을 때보다 성장했다는 것을 느끼게 되었고 C++ 의 몰랐던 기능들에 대해서 더 자세히 알 수 있었습니다.

체계적인 설계를 했을 때 전체적인 구조가 깔끔해지고 유지보수 하기에 편해진다는 것을 실제로 연습을 하면서 느끼게 되었습니다.

아직 구현하고 싶은 기술들이 많이 남아있고 앞으로도 많은 기술들을 생각하고 구현하면서 더 성장해야겠다는 다짐을 했습니다.

목차

1. 엔진 전체 구조 설명

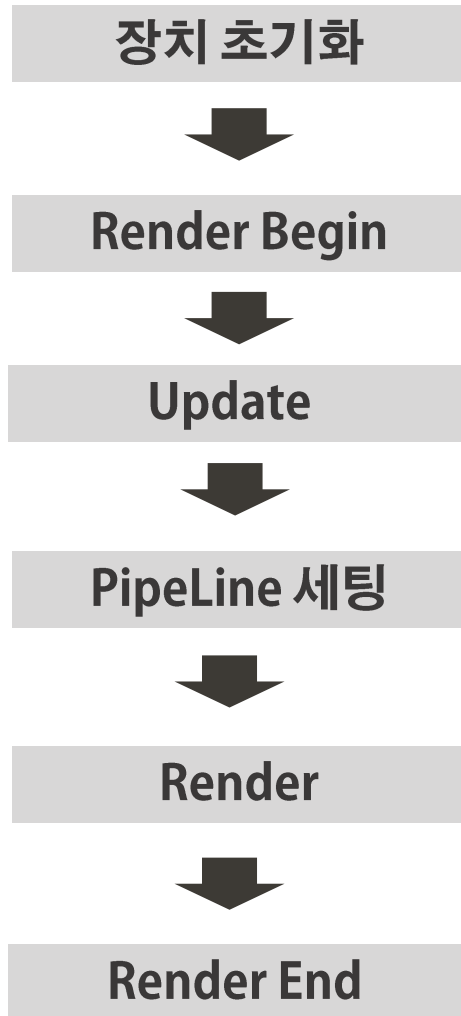
2. Update 과정

3. Render 과정

4. Deferred PBR

5. Multi Threading

6. Object Pool



각종 전역 데이터, 물체들 업데이트하는 단계



Object Manager

모든 물체들을 해당하는 파이프 라인에 따라 해시 Map으로 저장하고 관리해주는 클래스

Render 시에 Map 을 순회해서 현재 key 에 설정된 파이프라인을 세팅하고 해당 파이프라인에 해당하는 모든 물체를 그려줍니다.

그려주기 전에 Frustum Culling 으로 그려줄지 말지 결정

Object

기본적인 물체의 틀 클래스

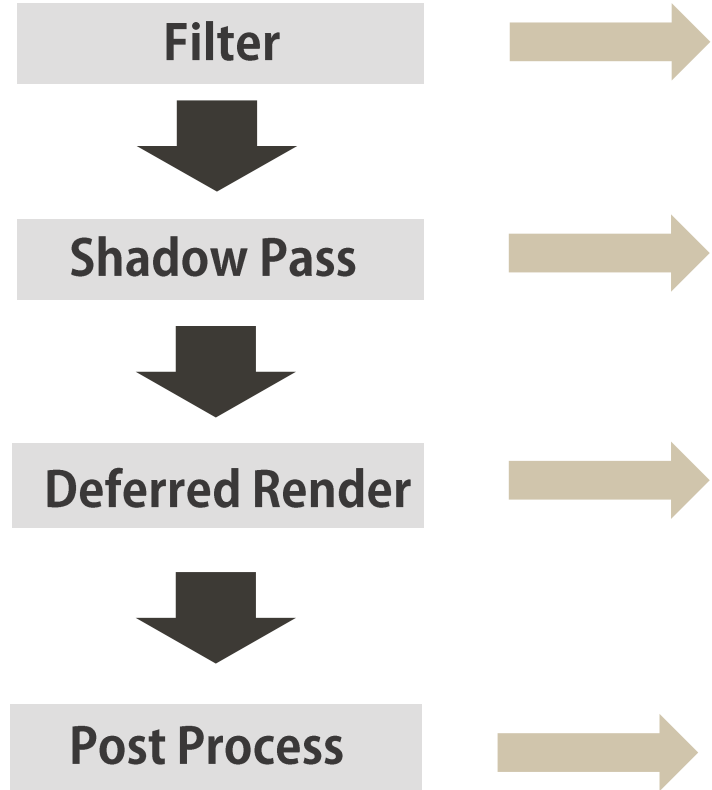
Component

물체가 가지고 있는 속성들을 추가
물체 업데이트 시 속성들 모두 업데이트

Material

물체가 가지고 있는 정보
Vertex , Index Buffer 및 CBV , SRV 를 설정하고 만들어 줍니다.

전역 데이터, 라이팅 바인딩하고
필터 렌더링 후
모든 물체들 그려주는 단계



Post Process 단계에서 계산해 줄 후처리 효과 및
픽셀 단위 정보를 저장하기 위한 텍스처 버퍼

Compute Shader 로 처리

그림자 깊이 맵 렌더링

3개의 렌더타겟을 세팅하여 Post Process 에서
라이팅 계산을 통해 최종 색상 결정

Deferred Rendering 에서 저장한 렌더타겟들을 바탕으로
라이팅 계산, 그림자 및 최종적으로 화면에 보여질
픽셀 후처리를 해주는 단계입니다.

PBR 을 하려면?

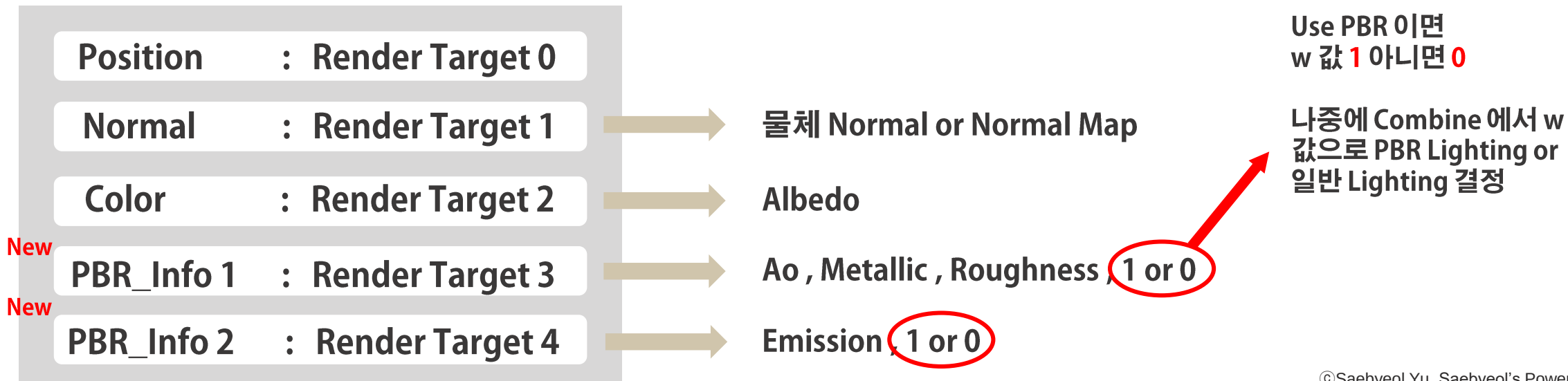
물체 Material 에 관한 정보들 (Albedo, Normal , Ao , Metallic, Roughness, Emission 등) 을 SRV 로 가져와서 Lighting 연산을 해주어야 합니다.

현재 Deferred Rendering 으로 픽셀마다 가지고 있는 정보를 Render Target 에 저장하고 있으므로 물체 Material 에 관한 정보들도 각각 Render Target 으로 저장해서 넘겨주는 방법

-> 최대 세팅 가능한 Render Target 개수 8개 이고 비효율적

해결방법

Render Target 을 두개만 더 만들어서 Material 정보 압축하기



SRV 레지스터 0번 ~ 5번에 바인딩 된 상황

```
Texture2D t_albedo      : register(t0);  
Texture2D t_normalMap   : register(t1);  
Texture2D t_ao          : register(t2);  
Texture2D t_metallic     : register(t3);  
Texture2D t_roughness    : register(t4);  
Texture2D t_emission     : register(t5);
```



여기서 Ao 텍스처를 사용하지 않는다면

```
Texture2D t_albedo      : register(t0);  
Texture2D t_normalMap   : register(t1);  
Texture2D t_ao          : register(t2);  
Texture2D t_metallic     : register(t3);  
Texture2D t_roughness    : register(t4);  
Texture2D t_emission     : register(t5);
```

t2
t3
t4

텍스처가 원하는 곳에 바인딩 되지 않고 하나씩 앞당겨서 잘못 바인딩 되는 상황이 발생할 수 있습니다.

이러한 상황을 발생시키지 않기 위해서 SRV Handle의 주소값을 알맞게 설정해주는 작업 필요

Object의 Texture를 추가할때마다 Texture는 미리 만들어 놓고

나중에 BeginPlay에서 각 Material Texture Flag를 통해 Texture를 사용하면

SRV를 만들고 아니면 SRV 주소 건너뛰고 다음 주소로 이동하는 방식으로

사용하는 텍스처의 주소값을 알맞게 설정해주고 있습니다.

Multi Threading 과정

List 에 담고 Queue 를 실행하는 과정을 여러 Thread 에 나눠 담아서 병렬로 실행합니다.

Custom Thread

std::thread

commandlist

싱글 쓰레딩

Shadow Pass



Deferred Render



PostProcess



Execute CommandList

싱글 쓰레딩에 사용되는 모든 List 와 Queue 는
CommandManager 에서 관리

멀티 쓰레딩 (Thread 2개)

Thread

Shadow Pass



Execute CommandList

Thread

Deferred Render



Execute CommandList



PostProcess



Execute CommandList

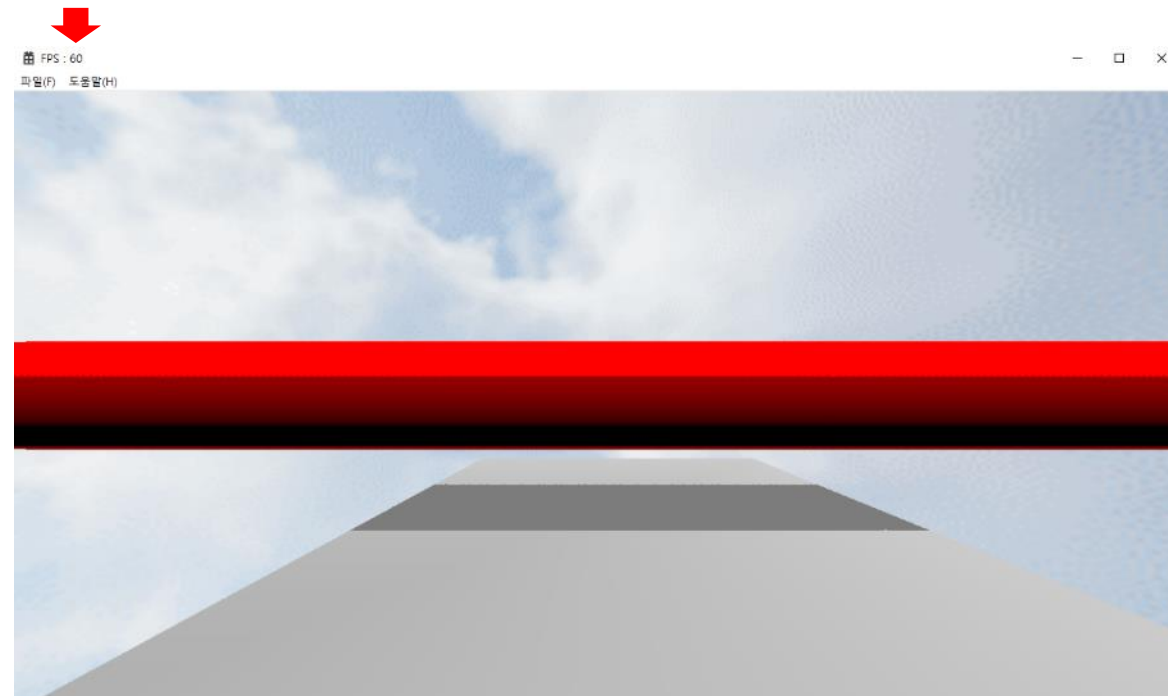
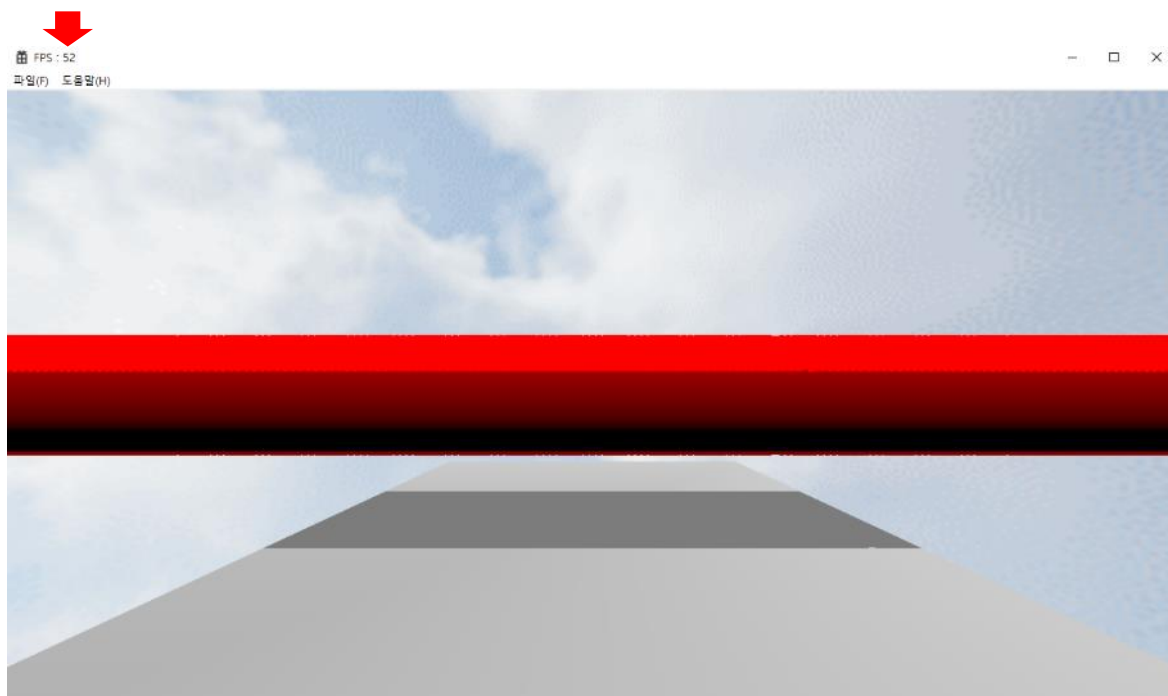
멀티 쓰레딩에 사용되는 모든 List 와 Queue 는
ThreadManager 에서 관리

Part 2 >> Multi Threading 결과

Vertex 개수	Single Threading FPS	Multi Threading FPS
2319개	905	360
23만 1900개	198	168
92만 7600개	92	96
185만5200개	52	60

그려주는 물체가 적으면
싱글 쓰레딩을 사용하는 것이
더 빠르고
물체가 많아질수록 멀티 쓰레딩의
성능이 점점 더 좋아집니다.

Pass 단위 대신 물체를 나눠담는
Thread
-> 렌더타겟 세팅, 자원상태전이 등
동기화 필요



Part 2 >> Object Pool

두개의 Map으로 Object Pool 관리

하나의 Map에 각 타입별 큐 할당해서 객체 미리 만들어 놓고 꺼내 쓰고 다 쓴 객체는 다른 Map에 있는 큐에 반납합니다.
꺼내 올 큐의 객체를 다 썼다면 반납 큐와 Swap 을 해주고 반납 큐에도 반납한 객체가 없다면 재할당을 해줍니다.

